Putting the Pieces Together

At this point, we have a subject, 1, that we've wrapped inside the expect method to make it testable. We also have a matcher, be_one. We can put them together using the to or not_to method:

```
>> expect_one.to(be_one)
=> true
>> expect_one.not_to(be_one)
RSpec::Expectations::ExpectationNotMetError:
expected: value != 1
     got: 1

(compared using ==)

    « backtrace truncated »
```

The to method tries to match the subject (in this case, the integer 1) against the provided matcher. If there's a match, the method returns true; if not, it bails with a detailed failure message.

The not_to method does the opposite: it fails if the subject *does* match. If you like to boldly split infinitives, you can also use to_not in place of not_to:

```
>> expect(1).not_to eq(2)
=> true
>> expect(1).to_not eq(2)
=> true
```

When you think of RSpec expectations as being just a couple of simple Ruby objects glued together, the syntax becomes clear. You'll use parentheses with the expect method call, a dot to attach the to or not_to method, and a space leading up to the matcher.

When an Expectation Fails

When the code you're testing does not behave as you expect, it's essential to have good, detailed error information to quickly diagnose what's going on. As you've seen, RSpec's matchers provide helpful failure messages right out of the box. Sometimes, though, you need a little more detail.

For example, consider the following expectation:

```
>> resp = Struct.new(:status, :body).new(400, 'unknown query param `sort`')
=> #<struct status=400, body="unknown query param `sort`">

>> expect(resp.status).to eq(200)
RSpec::Expectations::ExpectationNotMetError:
expected: 200
     got: 400
```